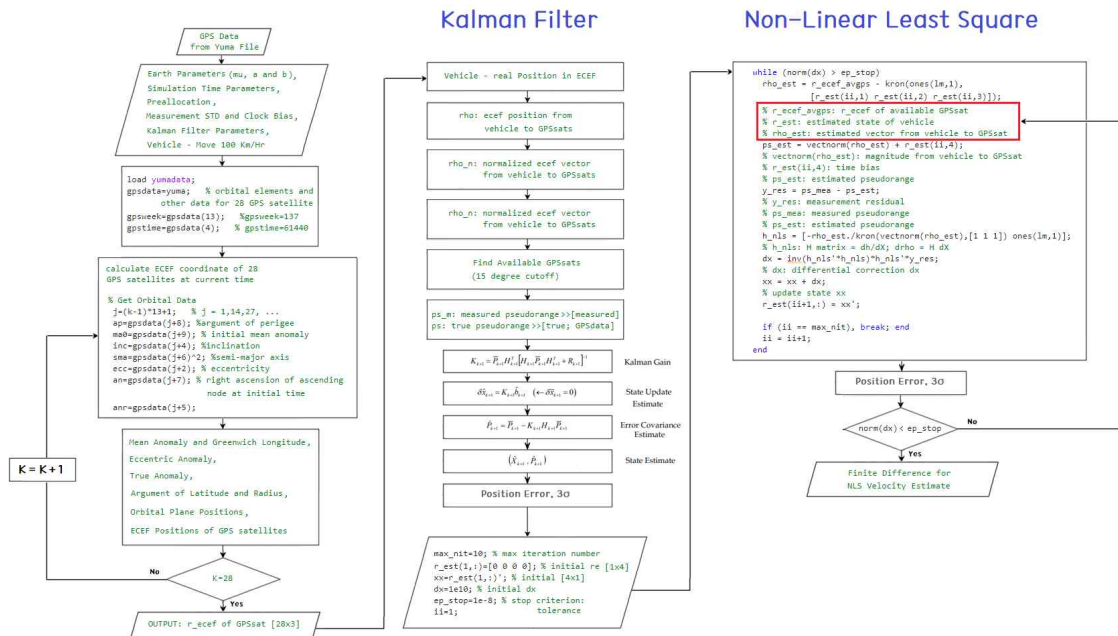


궤도결정과 필터링 HW#4

2022311224 이진진

1) Draw a flowchart for the program.



2) Analyze the program codes provided as detail as possible

① POD_HW4.m

: 이번 과제를 수행하는데 핵심이 되는 파일이다. POD의 과정에 대한 정보가 일괄적으로 입력되어 있으며 이에 대한 자세한 설명은 아래 첨부하였다.

```
편집기 - C:\Users\이진진\Downloads\숙제4_codes\숙제4_codes\POD_HW4.m
POD_HW4.m x POD_HW4_DGPS_submit.m x vectnorm.m x +

13 %% Dataload and Parameter Setting
14
15 % GPS Data from Yuma File Yuma file로부터 GPS 데이터를 불러온다
16 load yumadata;
17 gpsdata=yuma; % orbital elements and other data for 28 GPS satellite
18 gpsweek=gpsdata(13); %gpsweek=137
19 gpstime=gpsdata(4); % gpstime=61440 Week과 time에 대한 정보를 저장
20
21 % Earth Parameters (mu, a and b are in meters) 지구에 대한 정보를 불러온다
22 we=7.2921151467e-5; % rotation rate of earth [rad/s]
23 mu=398600.5e9; % gravitational const. of earth [m3/s2]
24 e=0.0818; % eccentricity of earth
25 a=6378137; % (equatorial) radius of earth [m]
26 b=6356752.3142; % (polar) radius of earth [m] (=a*(1-flat))
27 % flatness = (a-b)/a = 0.003352810664747
28
29 % Simulation Time Parameters 시뮬레이션 하기 위한 시간 정보를 저장
30 dt=10; % time interval
31 tf=3600; % final time
32 t=(0:dt:tf)'; % 0,10,20,...,3600 [361x1] 10초의 time interval로
33 m=length(t); % # of timestep (= 361) 3600초를 361개 구간으로 나눈다
34 i50=0;
35
36 % Preallocation
37 sig3=zeros(m,4); sig3_nls=zeros(m,4); % sigma [361x4]
38 lam=zeros(m,1); phi=zeros(m,1); h=zeros(m,1); % lam,phi,h [361x1]
39 pos_err=zeros(m,3); number_av=zeros(m,1); % pos_err,errnls,est [mx3]; num_avg [mx1]
40 pos_err_nls=zeros(m,3); pos_est=zeros(m,3);
41 % Preallocation에 필요한 함수를 0벡터로 정의
42 %% dontknow stt
43 % Measurement STD and Clock Bias in Meters
44 sigm=5; % measurement stdev = 5 (m)
45 bias=85000; % assumed clock bias = 85000 (m) (from ex.4.1)
46 qbias=200;
47 drift=lsim(0,1,1,0,sqrt(qbias)*randn(m,1),t); %LSIM Simulate time response of LTI mode:
48 % lsim(sys1,sys2,sys3,sys4,u,t)
49 % system: 0,1,1,0 STD 및 Clock Bias에 대한 함수를
50 % arbitrary input: sqrt(qbias)*randn(m,1) meter단위로 측정
51 % time: t
52
53 % Kalman Filter Parameters
54 ppos=1e6^2; pvel=100^2; pbias=100000^2; % position, velocity, clock bias (sigma^2)
55 pp=zeros(7); % [7x7]
56 pp(1,1)=ppos; pp(2,2)=ppos; pp(3,3)=ppos; % initial covariance for position (sigma^2)
57 pp(4,4)=pvel; pp(5,5)=pvel; pp(6,6)=pvel; % initial covariance for velocity (sigma^2)
58 pp(7,7)=pbias; % initial covariance for clock bias (sigma^2)
59 sig3(1,:)= [pp(1,1) pp(2,2) pp(3,3) pp(7,7)].^(0.5)*3; % position, clock bias (3sigma) %
60
61 phi_s=eye(7); % [7x7]
62 phi_s(1,4)=dt; phi_s(2,5)=dt; phi_s(3,6)=dt; % discrete-time version of system dynamics
63 x_est_veh=zeros(m,7); % assumption of initial states of vehicle on the earth [mx7]
64 qv=0.00001;
65 gammaq=[dt^2/2*eye(3) zeros(3,1); dt*eye(4)]; % [7x4]
66 qm=zeros(4); qm(1,1)=qv; qm(2,2)=qv; qm(3,3)=qv; qm(4,4)=qbias;
67 % %% dontknow end Kalman Filter에 이용할 parameter에 대한 함수
68
```

```

편집기 - C:\Users\이진진\Downloads\숙제4_codes\숙제4_codes\POD_HW4.m
POD_HW4.m x POD_HW4_DGPS_submit.m x vectnorm.m x +
69 %% Vehicle - Move 100 Km/Hr South from Washington, DC
70 deg_inc=((a+b)/2/1000)*2*pi/360; % [km/deg]
71 % (a+b)/2/1000: average radius(km)
72 % ~ *2*pi: radius*angle = arclength (km)
73 % ~ /360: arclength/inclination (km/deg)
74 south_inc=100/deg_inc/3600; % [deg/sec]
75 % 100 (km/hr) * 1/deg_inc (deg/km) * 1/3600 (hr/sec)
76 lambda=(38 : south_inc : 38+south_inc*(m-1))*pi/180; % [rad] % [1xm]
77 % cf) Coordinates of Washington, DC: 38°54'17"N 77°00'59"W
78 % (assumed) coordinate of vehicle: 38°N 77°W (from ex.4.1)
79 % lambda: latitude of vehicle; from Washington,DC(38°); toward south in 100km/hr;
80
81
82 %% Main Loop
83
84 % i == time step (dt=1 sec)
85 for i = 1:m-1 % i = 1:360
86
87     % Display When Every 50th Point is Reached
88     if (i50==50)
89         disp(sprintf('Program has reached point %5i',i-1))
90         i50=0;
91     end
92     i50=i50+1;
93
94     % Get GPS GPSsats Vectors
95     % Note the YUMA almanac gives the "longitude of the ascending
96     % node," which is referred to the Greenwich meridian. So, a
97     % sidereal angle computation is not needed.
98
99     % Matrix of Stored ECEF GPSsats Positions
100     r_ecef_gps=zeros(28,3); % [28x3]
101
102     % Get Time of Applicability and Current Time
103     tapp=gpsdata(4); % (= 61440)
104     tcurr=tapp+t(i); % tapp + 10*i
105
106     %%--- calculate ECEF coordinate of 28 GPS satellites at current time
107     % INPUT: gpsdata(orbital elements, time, etc.)
108     for k = 1:28
109
110         % Get Orbital Data
111         j=(k-1)*13+1; % j = 1,14,27, ...
112         ap=gpsdata(j+8); % argument of perigee
113         ma0=gpsdata(j+9); % initial mean anomaly
114         inc=gpsdata(j+4); % inclination
115         sma=gpsdata(j+6)^2; %semi-major axis
116         ecc=gpsdata(j+2); % eccentricity
117         an=gpsdata(j+7); % right ascension of ascending node at initial time
118         anr=gpsdata(j+5);
119
120         % Mean Anomaly and Greenwich Longitude
121         ma=ma0+sqrt(mu/(sma^3))*(tcurr-tapp); %Mean Anomaly at current time
122         om=an+anr*(tcurr-tapp)-we*tcurr; % sidereal angle
123
124         % Eccentric Anomaly (from "Celestial Mechanics", pg. 161) and Iteration
125         E=ma+ecc*sin(ma)+(ecc^2/2)*sin(2*ma)+(ecc^3/24)* ...
126             (9*sin(3*ma)-3*sin(ma))+((ecc^4/192)*(64 * ...
127             sin(4*ma)-32*sin(2*ma)));
128         while abs(E-ecc*sin(E)-ma)>1e-14
129             E=E-(E-ecc*sin(E)-ma)/(1-ecc*cos(E));
130         end
131

```

워싱턴 DC를 출발하여 남쪽으로 100km/h 속도로 움직이는 위성의 궤도를 계산하기 위한 수식

50이 되는 point를 기준으로 이를 반복 실행

이 과정을 반복수행

궤도
6요소

+ anr

ECEF의 좌표계에서의 28개의 GPS로부터
받은 데이터를 통해 위성의 궤도요소를 산출

mean anomaly와 greenwich의

위도에 대한 정보

이를 이용해 현재 시각에 대한 Mean anomaly,
Sidereal angle, Eccentric anomaly를 계산

```

132 % True Anomaly, Argument of Latitude and Radius
133 cta=(cos(E)-ecc)/(1-ecc*cos(E));
134 sta=sqrt(1-ecc^2)*sin(E)/(1-ecc*cos(E));
135 trueanon=atan2(sta,cta);
136 arglat=trueanon+ap; % argument of latitude
137 radius=sma*(1-ecc*cos(E));
138
139 % Orbital Plane Positions
140 x_prime=radius*cos(arglat); % x position on the orbital plane
141 y_prime=radius*sin(arglat); % y position on the orbital plane
142
143 % ECEF Positions of GPS satellites
144 x_coord=x_prime*cos(om)-y_prime*cos(inc)*sin(om); % (double)
145 y_coord=x_prime*sin(om)+y_prime*cos(inc)*cos(om); % (double)
146 z_coord=y_prime*sin(inc); % (double)
147 r_ecef_gps(k,:)=x_coord y_coord z_coord; % [28x3]
148 % r_ecef_gps(k,:): r_ecef of kth GPSsat >>[true; GPSdata]
149
150 end
151 %-- end of calculating ECEF coordinate of 28 GPS satellitesat current time
152 % % OUTPUT: r_ecef of GPSsat [28x3] >>[measured; true; z]
153
154 % % Vehicle - Washington DC (note: h is given in meters)
155 phi=(-77)*pi/180; % [rad] % longitude of vehicle
156 % cf) Coordinates of Washington, DC: 38°54'17"N 77°00'59"W
157 h=0;
158 n_lam=a/sqrt(1-e^2*sin(lambda(i))^2);
159 % length of the normal to the ellipsoid
160
161 % % Vehicle - real Position in ECEF
162 x_ecef_veh=(n_lam+h)*cos(lambda(i))*cos(phi); % (double)
163 y_ecef_veh=(n_lam+h)*cos(lambda(i))*sin(phi); % (double)
164 z_ecef_veh=(n_lam*(1-e^2)+h)*sin(lambda(i)); % (double)
165 % xyz_ecef_veh >> [assumed position of vehicle]
166
167 % % Vectors from vehicle to GPSsats
168 r_kron_veh = kron(ones(28,1),[x_ecef_veh y_ecef_veh z_ecef_veh]); % [28x3] % 28
169 % r_kron_veh: ecef position of vehicle
170
171 rho = r_ecef_gps - r_kron_veh; % [28x3]
172 % rho: ecef position from vehicle to GPSsats
173 % r_ecef_veh: ecef position of GPSsats
174
175 rhomag = vectnorm(rho); % [28x1] % mag of each row
176 % rhomag: magnitude from vehicle to GPSsats
177 rho_n = rho./kron(rhomag,[1 1 1]); % [28x3] % row-normalized rho
178 % rho_n: normalized ecef vector from vehicle to GPSsats
179
180 % % Find Available GPSsats (15 degree cutoff)
181 up = [cos(lambda(i))*cos(phi), cos(lambda(i))*sin(phi), sin(lambda(i))]; % upvector
182 zenang = acos(rho_n(:,1)*up(1)+rho_n(:,2)*up(2)+rho_n(:,3)*up(3))*180/pi; % zenith
183 seenang = 90 - zenang; % vertical(elevation) angle of GPSsats [deg]
184 av = find(seenang > 15); % Choose GPSsats if vertical angle > 15 % [?x1]
185 available_GPSsats = av'; % list of available GPSsat @t(i) % [1x?] % ex.4,7,9,17
186
187 r_ecef_avgps = r_ecef_gps(av,:); % r_ecef of available GPSsats @t(i) % [?x3]
188 % >>[true; GPSdata]
189 ps = rhomag(av,:); % rhomag of available GPSsats @t(i) % [?x1]
190 % >>[true; GPSdata]
191
192 lm = length(ps); % number of available GPSsats @t(i)
193 number_av(i) = lm; % number of available GPSsats @t(i)
194
195 % % Measurements
196 ps_mea = ps + sigm*randn(lm,1) + bias*ones(lm,1) + drift(i)*ones(lm,1);
197 % ps_m: measured pseudorange % >>[measured]
198 % ps: true pseudorange % >>[true; GPSdata]
199 % randn = random number generation w/ standard normal distribution;
200 % (i.e. zero mean; unit variance(sigma^2=1))
201 % sigm*randm: random noise
202 % bias: position bias
203 % drift: drift due to clock bias

```

True anomaly, argument of latitude, Radius를 계산하는 함수

궤도면에서의 위치를 계산

이를 위해 위에서 계산한 radius, arg, lat을 이용한다

위의 계산한 궤도면의 위치를 통해 GPS의 위치벡터를 계산 (ECEF좌표계)
ECEF GPS의 위치벡터가 28x3 크기로 출력된다
위성체의 위치를 저장한다
(위성체의 관측지 위치 - 38N, 77W)

저장된 실제 위치 정보를 ECEF의 새 축에 대해 표현

위성체에서 GPS까지의 거리를 계산

거리(rho)를 normalize
15도를 기준으로 관측지에서

15 degree cutoff) 위성이 관측 가능한지 여부를 판단

이를 위해 upvector, zenith angle, seeing angle을 계산

cut off 각도가 15도 이상인 위성에 대해 pseudorange를 추정

```

207 % -- Kalman Filter Routine  위의 정보들을 이용해 Kalman Filter를 수행
208 x_est_veh(1,:) = [x_ecef_veh y_ecef_veh z_ecef_veh 0 0 0]; % [mx7]
209 % x_est_veh(1,:): initial guess; >>[predicted; bar] 초기 x값을 위와 같이 설정
210
211 rho_est = r_ecef_avgps - kron(ones(1m,1),[x_est_veh(i,1) x_est_veh(i,2) x_est_veh(i,3)]);
212 % r_ecef_avgps = r_ecef of available GPSsats
213 % x_est_veh(i,1:3) = r_ecef of vehicle >>[predicted; bar]
214 % rho_est = r_ecef from vehicle to GPSsats ECEF에서의 rho를 계산하는 함수
215
216 ps_est = vectnorm(rho_est) + x_est_veh(i,7);
217 % ps_est: estimated pseudorange
218 % vectnorm(rho_est): magnitude from vehicle to GPSsat
219 % x_est_veh(i,7): clock bias pseudorange를 계산
220
221 y_res = ps_meas - ps_est;
222 % y_res = tilde(b) % measurement residual % [measured - estimated]
223 % ps_meas = z >>[measured; z]
224 % ps_est = H*bar(X) >>[estimated]
225 hh = [-rho_est./kron(vectnorm(rho_est),[1 1 1]), ones(1m,1)]; % [1mx4] % [-rho_ecef, 1, 0, 0, 0, 0, 1]
226 hh = [hh(:,1) hh(:,2) hh(:,3) zeros(1m,3) hh(:,4)]; % [1mx7] % [-rho_norm, 0, 0, 0, 1, 0, 0]
227 % H matrix pseudorange의 측정값과 추정치의 차를 이용하여 H matrix를 계산
228
229 % % Update
230 gain = pp*hh'*inv(hh*pp*hh'+sigm^2*eye(1m));
231 % gain: Kalman gain
232 % pp: covariance matrix >>[predicted; bar]
233 % hh: H matrix Covariance matrix와 H matrix에 대한 식을 통해
234 % sigm^2: R matrix Kalman gain을 위와 같이 계산
235
236 del_x_est_veh = (gain*y_res)';
237 % state update >>[estimated; hat] Kalman gain에 관측된 residual 값을 곱하여
238 % delta x를 계산
239 x_est_veh(i,:) = x_est_veh(i,:) + del_x_est_veh; % state update estimate (estimated)
240 % update state(position of vehicle) >>[estimated; hat]
241 pp = (eye(7)-gain*hh)*pp; %covariance estimate
242 % updated covariance matrix >>[estimated; hat] 계산된 관측 값을 새롭게 update
243
244 % % Propagate 위의 정보를 통해 다음 시각으로의 전파
245 x_est_veh(i+1,:) = (phi_s*x_est_veh(i,:))'; % propagate states of the vehicle on t
246 % propagate state from t(i) to t(i+1)
247 % x_est_veh(i,:): estimated state at t(i) >>[estimated; hat]
248 % phi_s: state transition matrix
249 % x_est_veh(i+1,:): predicted state at t(i+1) >>[predicted; bar]
250 pp = phi_s*pp*phi_s' + gammaq*gm*gammaq'; % propagate covariance to next time step
251 % propagate covariance matrix >>[predicted]
252 % pp(RHS): estimated covariance at t(i) >>[estimated; hat] 다음 시각에서의
253 % pp(LHS): predicted covariance at t(i+1) >>[predicted; bar] x값을 예측
254
255 % % Calculate position error and 3sigma
256 pos_err(i,:) = x_est_veh(i,1:3) - [x_ecef_veh y_ecef_veh z_ecef_veh];
257 % x_est_veh(i,1:3): estimated state at t(i) >>[estimated; hat]
258 % xyz_ecef_veh: >>[true (of vehicle)]
259 % pos_err: >>[estimated - true] 추정값과 참값의 차이를 이용하여 오차를 계산
260
261 sig3(i+1,:) = abs([pp(1,1) pp(2,2) pp(3,3) pp(7,7)]).^0.5*3;
262
263 위의 공식을 통해 표준편차를 구하고 3을 곱하여 3σ를 계산

```

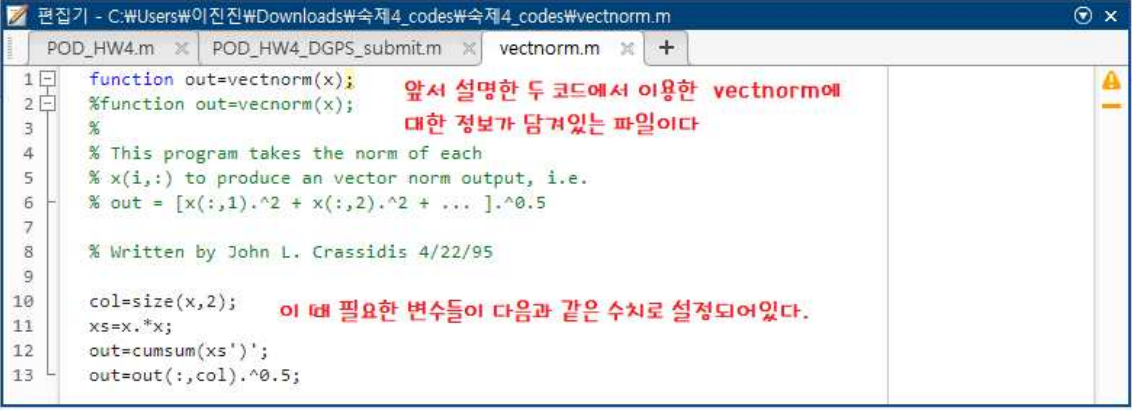
```

264 % -- Nonlinear Least Squares Nonlinear Least square를 계산하는 함수
265 max_nit=10; % max iteration number
266 clear re
267 r_est(1,:)=[0 0 0 0]; % initial re [1x4]
268 xx=r_est(1,:); % initial [4x1]
269 dx=1e10; % initial dx
270 ep_stop=1e-8; % stop criterion: tolerance
271 ii=1;
272
273 while (norm(dx) > ep_stop) norm(dx)가 tolerance보다 작아질때까지 while 반복문을 수행
274     rho_est = r_ecef_avgps - kron(ones(1m,1),[r_est(ii,1) r_est(ii,2) r_est(ii,3)]);
275     % r_ecef_avgps: r_ecef of available GPSsat
276     % r_est: estimated state of vehicle vehicle부터 위성까지 거리벡터를 추정
277     % rho_est: estimated vector from vehicle to GPSsat
278     ps_est = vectnorm(rho_est) + r_est(ii,4); rho의 추정값을 통해 pseudorange를 추정
279     % vectnorm(rho_est): magnitude from vehicle to GPSsat
280     % r_est(ii,4): time bias
281     % ps_est: estimated pseudorange
282     y_res = ps_meas - ps_est; pseudorange의 측정값에서
283     % y_res: measurement residual 위에서 구한 추정 값을 빼서 residual을 계산
284     % ps_meas: measured pseudorange
285     % ps_est: estimated pseudorange
286     h_nls = [-rho_est./kron(vectnorm(rho_est),[1 1 1]) ones(1m,1)];
287     % h_nls: H matrix = dh/dX; drho = H dX H matrix
288     dx = inv(h_nls'*h_nls)*h_nls'*y_res;
289     % dx: differential correction dx delta x
290     xx = xx + dx;
291     % update state xx
292     r_est(ii+1,:) = xx'; X의 추정치를 계산
293
294     if (ii == max_nit), break; end
295     ii = ii+1;
296 end
297
298 pos_est(i,:) = xx(1:3)'; % state >>[estimated]
299 pos_err_nls(i,:) = xx(1:3)'-[x_ecef_veh y_ecef_veh z_ecef_veh]; % >>[estimated - ]
300 % pos_err_nls: Position error of NLS method
301 pcov_nls = inv(h_nls'*h_nls)*sigm^2; % (sigma^2)
302 % pcov_nls: covariance P of NLS method
303 sig3_nls(i,:) = diag(pcov_nls)'.^(0.5)*3; % (3sigma)
304 % sig3_nls: 3 sigma of NLS method
305
306 end EKF와 동일한 방법으로 position error 및 3σ를 계산
307
308 % Finite Difference for NLS Velocity Estimate
309 vel_est = diff(pos_est)/dt;
310 % diff: calculate difference between adjacent element of pos_est
311 % dt: time interval
312 vel_est(m,:) = vel_est(m-1,:); 추정된 position vector를 interval time으로
313 % 나누어 해당 구간의 속도벡터를 계산
314
315 %% Plot Results
316 % figure(1) 이후부터는 위의 모든 과정에 대한 내용을
317 figure(1) plot으로 표현하기 위한 함수이다.
318 subplot(311)
319 plot(t/60,[-sig3(:,1) pos_err(:,1) sig3(:,1)]);
320 axis([0 60 -20 20])
321 set(gca,'fontsize',12)
322 set(gca,'Ytick',[-20 -10 0 10 20]);
323 set(gca,'Xtick',[0 10 20 30 40 50 60]);
324 ylabel('EKF x_err (m)')
325 legend('-3\sigma','x_{err}','+3\sigma')
326 % xlabel('Time (Min)')

```

② vectnorm.m

: 위에서 설명한 코드에 필요한 vectnorm에 대한 정보가 담겨있다. column의 크기 및 xs의 정의가 다음과 같이 표현되어 있다.



```
1 function out=vectnorm(x);
2 %function out=vecnorm(x);
3 %
4 % This program takes the norm of each
5 % x(i,:) to produce an vector norm output, i.e.
6 % out = [x(:,1).^2 + x(:,2).^2 + ... ].^0.5
7
8 % Written by John L. Crassidis 4/22/95
9
10 col=size(x,2);
11 xs=x.*x;
12 out=cumsum(xs')';
13 out=out(:,col).^0.5;
```

앞서 설명한 두 코드에서 이용한 vectnorm에 대한 정보가 담겨있는 파일이다

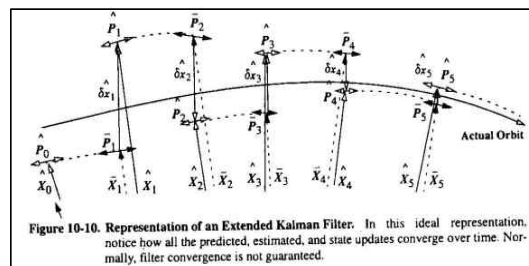
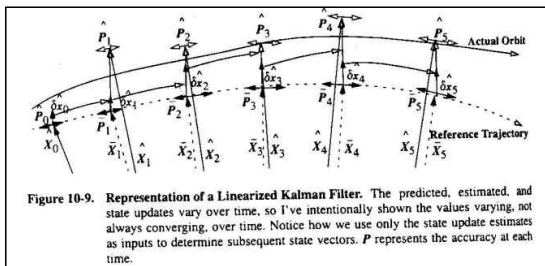
이 때 필요한 변수들이 다음과 같은 수치로 설정되어있다.

3) Explain the process of EKF

Kalman Filter는 $\hat{X}_k$ (이전 시각의 state estimate), $\hat{P}_k$ (정확도 나타내는 Covariance Matrix), $z_{k+1}(obs)$ (현재 시각에 관측된 값), Q (Dynamics와의 Error), R_{k+1} (현재 시각 관측값의 error) 등의 조건이 주어지면 그 다음 시각의 $\hat{X}$ 와 $\hat{P}$ 를 알 수 있게 해주는 측정 기법이다. Kalman Filter의 장점은 계산이 빠르고 요구되는 메모리 용량이 적으며, 현재 시각에 실시간으로 State와 Covariance Matrix를 추정할 수 있다는 점이 있다. 또한, Batch Filter와 달리 시간이 변화하는 것을 고려하여 실시간으로 계산하므로 매 시간마다 변화하는 변수들을 반영할 수 있다. Kalman Filter의 종류에는 세 가지가 있으며 LKF(Linear Kalman Filter), EKF(Extended Kalman Filter), UKF(Unscented Kalman Filter)이다.

가장 먼저, LKF(Linear Kalman Filter)는 특정 State를 기준으로 하여 새로운 Reference trajectory를 만들어 내며 이를 바탕으로 propagation을 한다. Reference trajectory와 actual orbit 간의 차이를 계산하는 방식으로 위치를 추정하게 되며 첫 state부터 propagation을 이어 나가기 때문에 시간이 지남에 따라 오차가 누적될 수 있다. 따라서 최종 궤도를 계산하였을 때 actual orbit과의 차이가 많이 발생할 수 있다는 치명적인 단점이 있다. 이 단점을 보완하기 위해 사용하는 Filtering 방식이 EKF(Extended Kalman Filter)이다.

EKF에서는 관측된 지점마다 새로운 Trajectory를 생성하는 방식을 이용하여 오차가 누적되는 것을 방지할 수 있다. LKF에서의 $\hat{X}$ 와 $\hat{P}$ 는 $\delta \bar{x}_{k+1}=0$ 을 기준으로 하지만 EKF에서는 현재 시각에 대해 update하며 매번 새로운 Reference Trajectory를 계산할 수 있다. 일련의 과정에 차이가 있음을 이해하기 위해 두 사진을 첨부하여 비교해보았다. (순서대로 LKF, EKF)



Algorithm: Extended Kalman Filter

Given $(\hat{X}_k, \hat{P}_k, Q, R_{k+1}, z_{k+1}(obs))$ 초기 조건을 전파하여

Want to know $\hat{X}_{k+1}, \hat{P}_{k+1}$ 현재의 $\hat{X}$ 와 $\hat{P}$ 를 계산하는 것이 EKF이다.

$$H = \frac{\partial g(X)}{\partial X} \Big|_{\bar{X}_{k+1}}$$

observation model $y = g(X)$

Partial Derivative Matrix는 H로 정의하며 위의 식과 같이 표현할 수 있다.

Prediction Prediction과 Update를 반복하여 매 시각 새로운 관측 값을 수정해나간다.

PKEPLER($\hat{X}_k, t_{k+1} - t_k \Rightarrow \bar{X}_{k+1}$) ; Predicted State

repeat each step

→ get new reference trajectory

동력학 모델을 이용하여 기존의 추정 값을 전파하여 계산된 state를 얻는다.

위 과정으로 새로운 reference trajectory를 얻을 수 있다.

$$F(t) = \frac{\partial \hat{X}_{k+1}}{\partial \hat{X}_k} \approx \frac{\partial \bar{X}_{k+1}}{\partial \bar{X}_k}$$

Because we can't know estimated state

추정된 state를 알 수 없기 때문에

계산한 state를 통해 근사한다.

$\Phi(t_{k+1}, t_k) = F(t)\Phi(t_{k+1}, t_k)$ 다음 식을 통해 State Transition Matrix를 계산할 수 있다.

→ Using numerical integration get $\Phi(t_{k+1}, t_k)$

$\delta \bar{X}_{k+1} = 0$; predicted state update

계산된 Matrix로 Predicted state와

$$\bar{P}_{k+1} = \Phi \hat{P}_k \Phi^T + Q$$

; predicted error covariance Covariance Matrix를 구할 수 있다.

Update

$$\tilde{b}_{k+1} = z_{k+1} - H_{k+1} \bar{X}_{k+1}$$

Kalman 식에 의해 residual을 다음과 같이 정의한다.

$$K_{k+1} = \bar{P}_{k+1} H_{k+1}^T [H_{k+1} \bar{P}_{k+1} H_{k+1}^T + R_{k+1}]^{-1} ; \text{Kalman Gain}$$

$$\delta \hat{X}_{k+1} = K_{k+1} \tilde{b}_{k+1} \quad (\leftarrow \delta \bar{X}_{k+1} = 0) ; \text{State Update Estimate}$$

$$\hat{P}_{k+1} = \bar{P}_{k+1} - K_{k+1} H_{k+1} \bar{P}_{k+1} ; \text{Error Covariance Estimate}$$

$$\hat{X}_{k+1} = \bar{X}_{k+1} + \delta \hat{X}_{k+1} ; \text{State Estimate}$$

$$\begin{aligned} \delta \hat{x}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} (A_{new}^T W_{new} \tilde{b}_{new} + A_k^T W_k \tilde{b}_k) \\ \hat{P}(0|k+n) &= (A_{new}^T W_{new} A_{new} + \hat{P}_k^{-1})^{-1} \end{aligned}$$

위의 식을 변형하여 Kalman gain, Error Covariance, State Estimate를 순차적으로 계산할 수 있다.

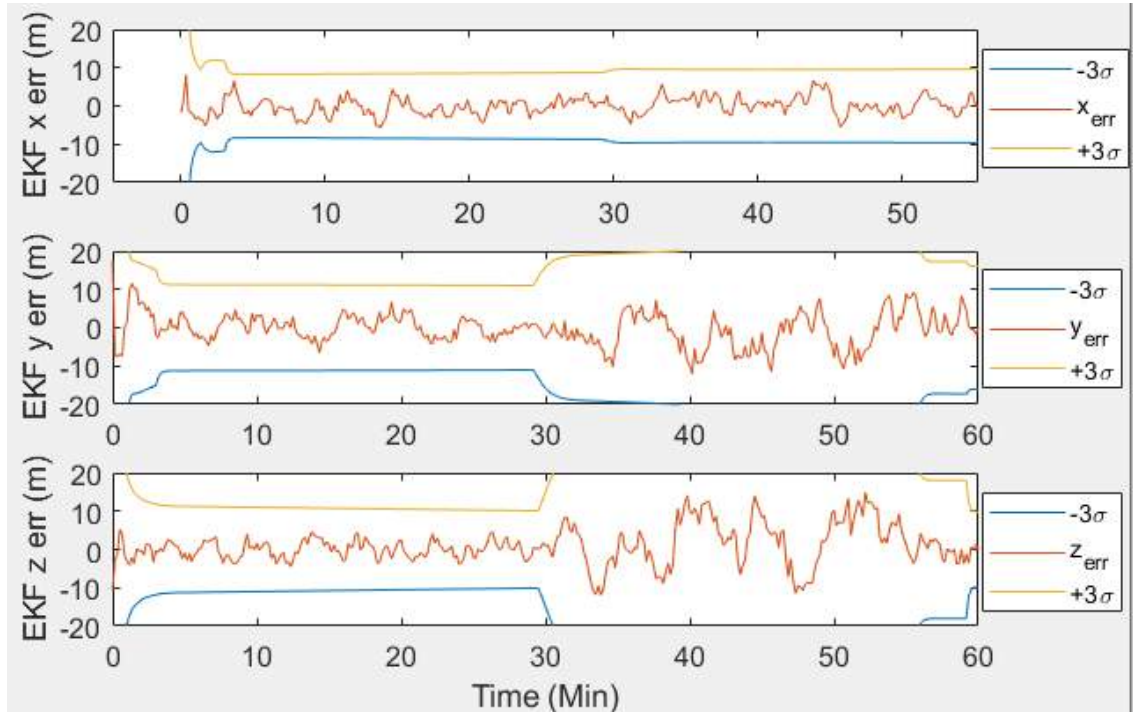
Example 7-1 procedure c)에 의하면, 관측지로부터 GPS의 elevation이 15도 이상인 조건이 있으므로 아래와 같은 식을 통해 E_i 의 각도를 확인 후에 $\hat{X}$ 와 $\hat{P}$ 를 추정해야 한다.

$$\rho_{UNE} = \begin{bmatrix} \cos \lambda & 0 & \sin \lambda \\ 0 & 1 & 0 \\ -\sin \lambda & 0 & \cos \lambda \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \rho$$

$$u = \begin{bmatrix} \cos \lambda \cos \phi \\ \cos \lambda \sin \phi \\ \sin \lambda \end{bmatrix} \quad \cos \xi_i = \rho_i^T u$$

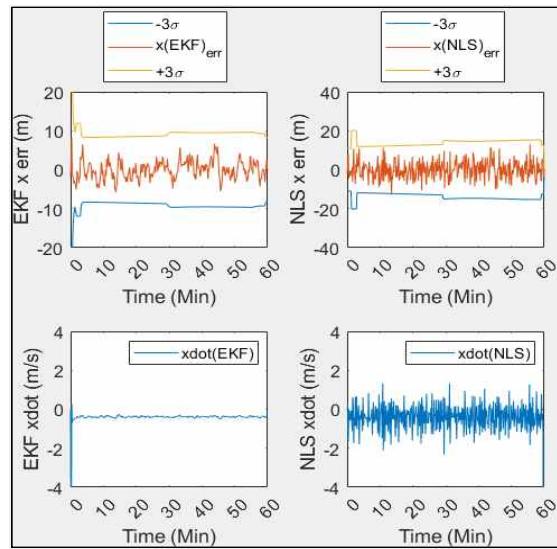
$$\rho_i = \frac{e_i - r}{\|e_i - r\|}, \quad E_i = 90^\circ - \xi_i$$

4) Explain the results obtained by EKF.

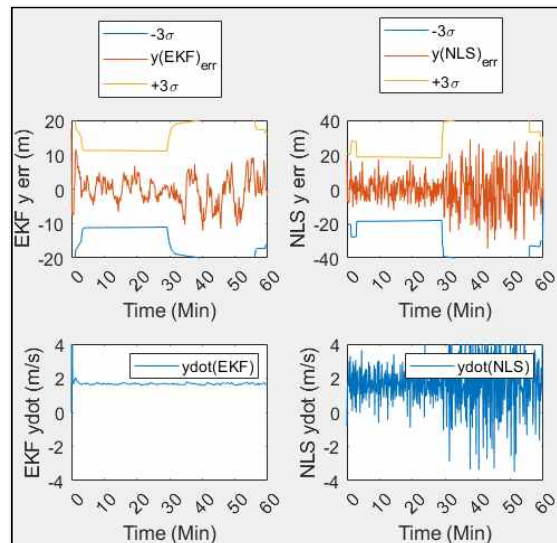


주어진 코드에 따라 EKF를 이용하여 simulation을 실행해본 결과 x , y , z 에 대한 position vector error가 위와 같이 출력되었다. 불규칙적으로 진동을 하는 position vector error를 기준으로 앞서 설명한 3-sigma가 위아래로 분포되어 있는 것을 확인할 수 있다. 일정하게 표준편차가 유지되는 x 축과는 달리 y , z 축에서는 30분이 되는 시점을 기준으로 $\pm 20\text{m}$ 위아래로 급격히 증가 또는 감소하는 모습을 볼 수 있다.

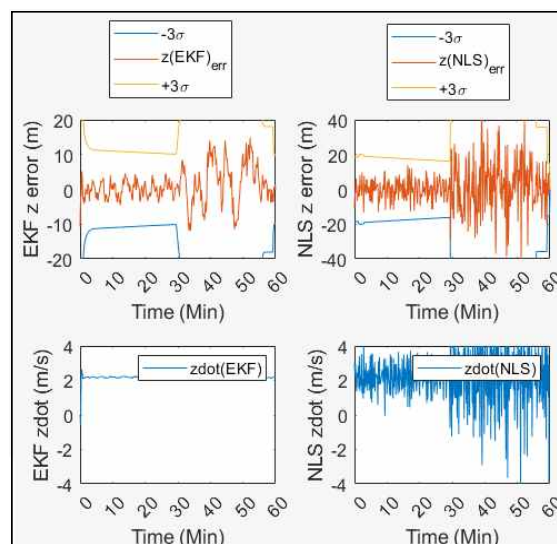
이 결과와 Least Nonlinear Square method를 통해 simulation한 결과에 어떤 차이가 있는지 알아보기 위해 x , y , z 축에 따른 두 그래프를 아래 그림과 같이 비교하였다. 동일한 조건에서의 error 값을 비교하기 위해 y 축 scale을 고정시킨 후 그래프를 배치한 모습이다.



EKF와 NLS의 Position error 비교 사진 (x축)



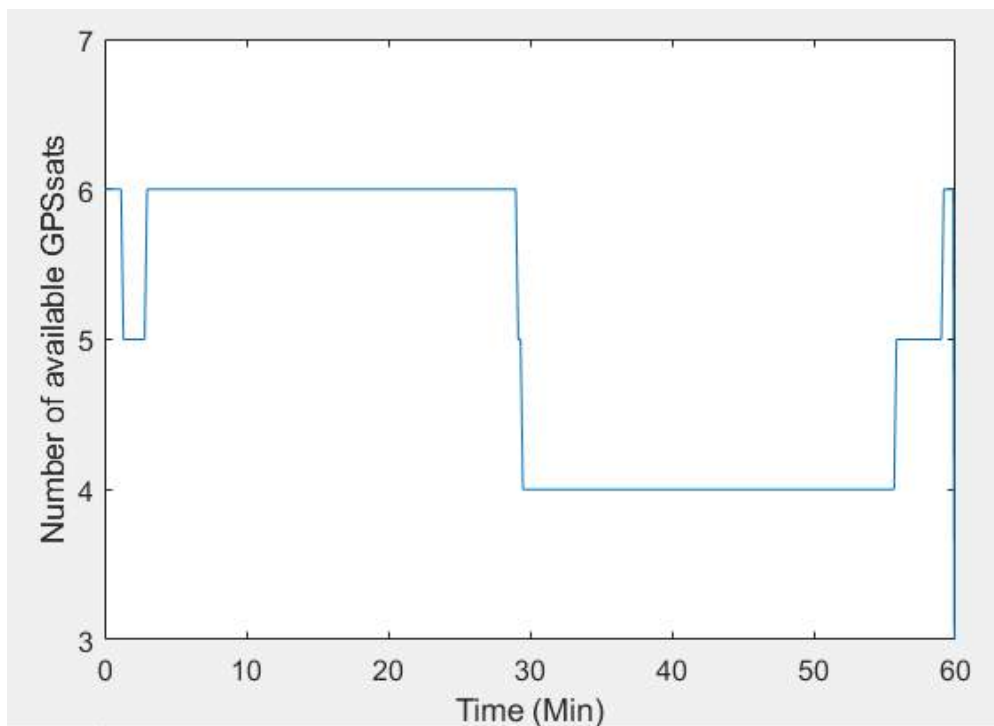
(y축)



(z축)

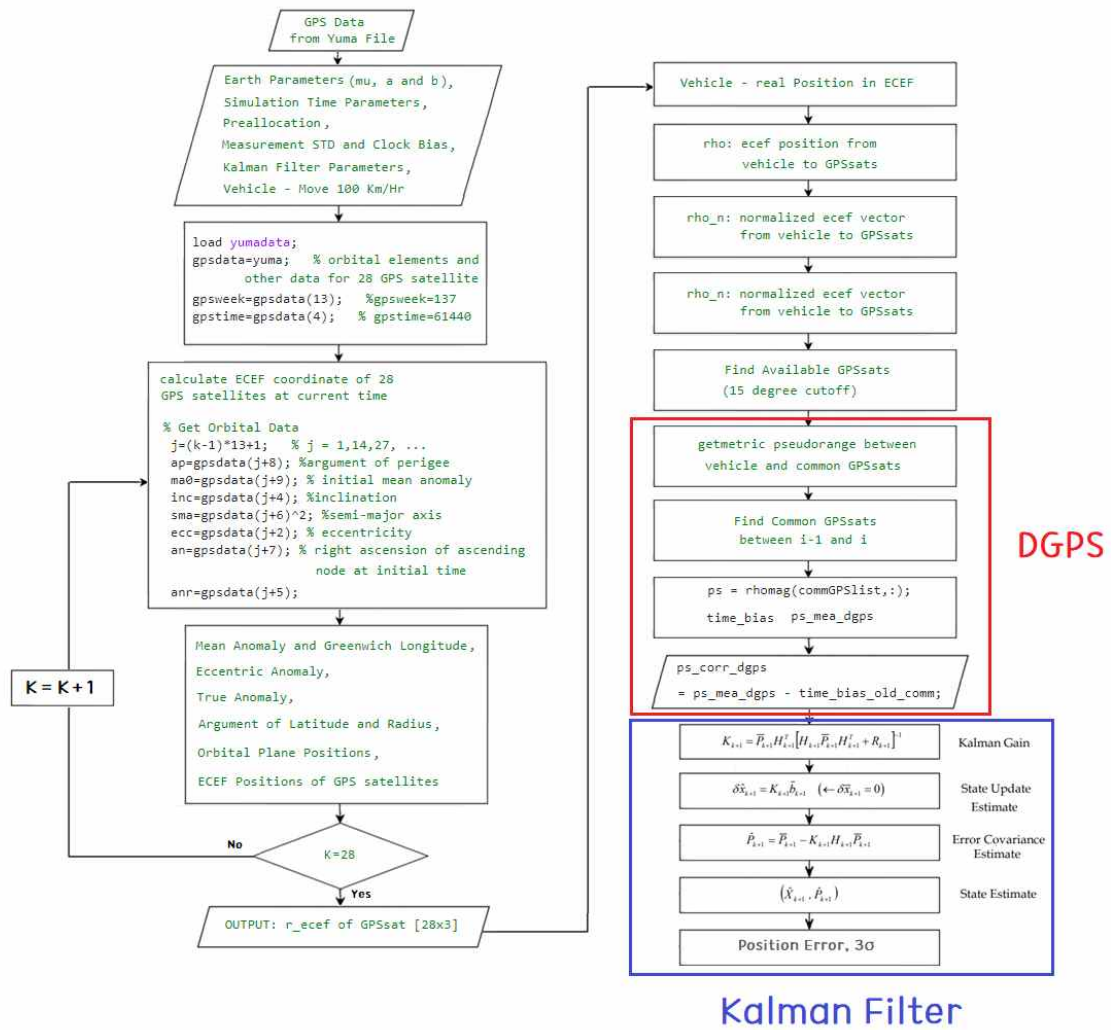
EKF에 의한 simulation 결과(왼쪽)와 Non Linear Square에 의한 결과(오른쪽)를 나타낸 그래프이며 위에서부터 순서대로 x, y, z축에 대한 position error의 값이다. 3σ 에 해당하는 slope는 두 방법에서 대체적으로 비슷하게 나타났지만 그 사이에 있는 position error slope는 모든 축에 대하여 Non Linear Square에 의한 결과에서 더 큰 진동을 보이며 특히 30분이 지난 시점에서는 표준편차와 함께 확연하게 진동의 폭이 증가함을 볼 수 있었다.

또한 velocity vector($\dot{x}$, $\dot{y}$, $\dot{z}$)에 대한 오차의 경우, EKF 방법을 사용했을 때는 비교적 일정한 모습을 보이고 있지만 Non Linear Square에서는 크게 요동치고 있는 것을 확인할 수 있다. 이 폭은 position error의 결과와 마찬가지로 simulation 실행 후 약 30분이 되는 시점에서 눈에 띄게 늘어나고 있다. 어떠한 이유로 이 구간에서 급격한 변화가 발생하는지 확인하기 위하여 시간에 따른 이용 가능한 GPS 위성의 수 그래프를 살펴보았다.



위의 그래프를 보고 이용 가능한 GPS의 수를 확인해보면 position 및 velocity error의 진동이 급격하게 증가하는 30분 구간부터 4대의 GPS만이 이용 가능한 것을 확인할 수 있다. 이는 이용 가능한 GPS 수가 많을수록 position 및 velocity error가 적으며 안정한 상태를 유지한다는 것을 유추해볼 수 있다.

5) Repeat the process of EKF by using DGPS.



6) Verify and Explain the DGPS code provided.

DGPS(Differential GPS)는 전국에 분포된 기준국에서 GPS 오차 성분 관측을 통해 보정정보를 생성하고 사용자에게 방송하여 위치오차 1m 이내의 정확한 위치를 알 수 있게 해주는 시스템이다. 문제 5)의 flowchart에서도 알 수 있듯이 DGPS는 Extended Kalman Filter를 이용하여 관측지의 위치 벡터를 추정하는 방법과 유사하게 수행한다. 현재 상태(i)에서의 이용 가능한 GPS의 수를 찾고 이전 상태(i-1)를 추정할 때 사용된 GPS의 교집합을 찾아 저장한 후 pseudo range를 계산한다. 그 후에 다음과 같은 공식을 통해 time bias를 계산한다. $\text{sigm} \cdot \text{randn}(\text{lm}, 1) + \text{bias} \cdot \text{ones}(\text{lm}, 1) + \text{drift}(i) \cdot \text{ones}(\text{lm}, 1)$ 마지막으로 이 두 값의 합으로 pseudo range의 관측값을 저장한다.

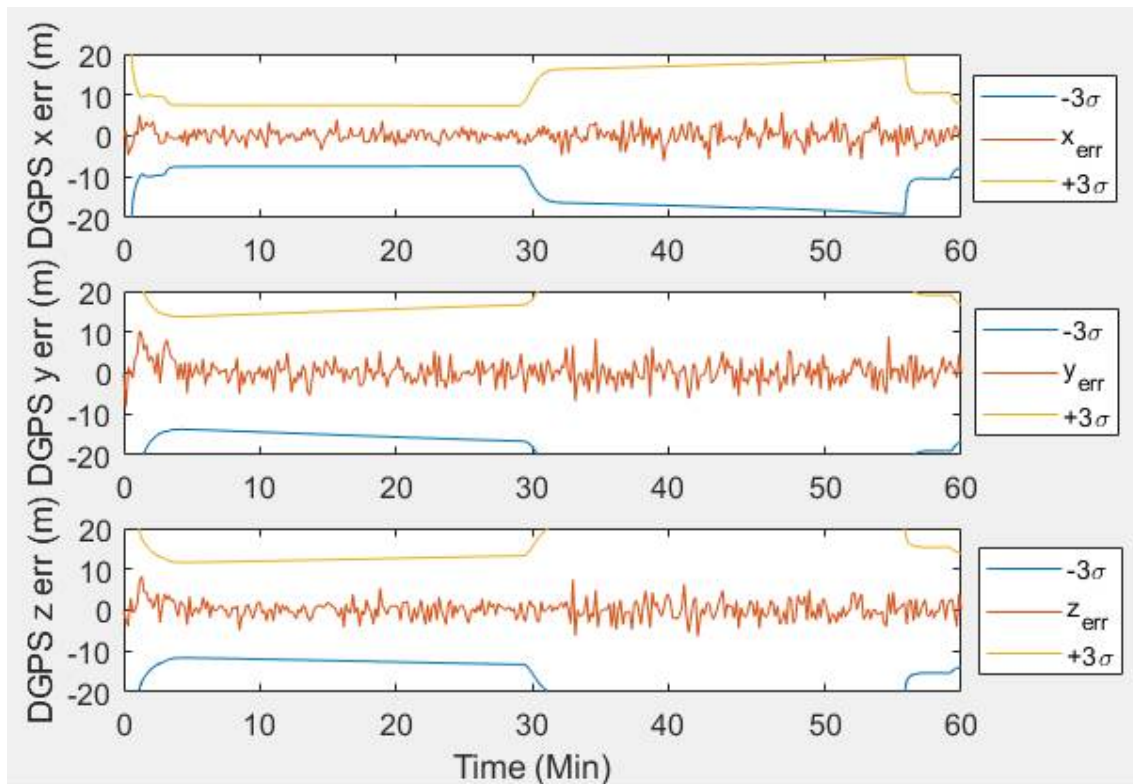
```

164
165 % Find Common GPSsats between i-1 and i common GPS 위성을 찾기 위해 반복문을 이용한다
166 if i == 1
167     available_GPSsats_old = available_GPSsats;
168 end
169 [commGPS,temp,commGPSlist] = intersect(available_GPSsats_old, available_GPSsats);
170 % commGPS: common GPS name % ex. 4,7,9,17,25
171 % temp: location of common GPS in 'available_GPSsats_old'
172 % commonGPSlist: location of common GPS in 'available_GPSsats' % ex. 1,2,3,4,6
173 lm = length(commGPSlist); % number of common GPS bet i-1 and i
174 num_commGPS(i) = lm; % save number of common GPS @t(i)
175 i와 i-1번째 사이에서 이용가능한 GPS의 교집합을 계산하여 저장한다
176 r_ecef_avgps = r_ecef_gps(commGPSlist,:); % r_ecef of common GPS at t(i)
177
178 ps = rhomag(commGPSlist,:); % getmetric pseudorange between vehicle and common GPSsats
179 time_bias = sigm*randn(lm,1) + bias*ones(lm,1) + drift(i)*ones(lm,1);
180 ps_meas_dgps = ps + time_bias; t(i)에서의 교집합 GPS의 위치벡터를 계산하고
181 vehicle와 위성 사이의 pseudorange 값을 계산한다.
182 % -- Kalman Filter Routine
183 x_est_veh(1,:) = [x_ecef_veh(i) y_ecef_veh(i) z_ecef_veh(i) 0 0 0 0]; % [mx7]
184
185 rho_est = r_ecef_avgps - kron(ones(lm,1),[x_est_veh(i,1) x_est_veh(i,2) x_est_veh(i,3)]);
186
187 ps_est = vectnorm(rho_est) + x_est_veh(i,7);
188
189
190
191 if i == 1
192     time_bias_old = time_bias; 계산된 pseudorange의 값을 보정하는 작업이다
193 end
194
195 if length(temp) > length(time_bias_old)
196     l_temp = length(temp);
197     l_bias = length(time_bias_old);
198     add = l_temp - l_bias;
199     time_bias_old = [time_bias_old; time_bias_old(1:add)];
200 end
201 time_bias_old_comm = time_bias_old(temp); 측정된 pseudorange값과 이전 시간의
202 ps_corr_dgps = ps_meas_dgps - time_bias_old_comm; 오차의 차를 이용하여 계산할 수 있다.
203
204 y_res = ps_corr_dgps - ps_est; 계산된 보정 값에서 추정된 ps 값을 빼면 residual이 계산된다
205
206 hh = [-rho_est./kron(vectnorm(rho_est),[1 1 1]), ones(lm,1)]; % [lmx4] % [-rho_norm
207 hh = [hh(:,1) hh(:,2) hh(:,3) zeros(lm,3) hh(:,4)]; % [lmx7] % [-rho_norm,0,0,0,1]
208
209 gain = pp*hh'*inv(hh*pp*hh'+sigm^2*eye(lm));
210

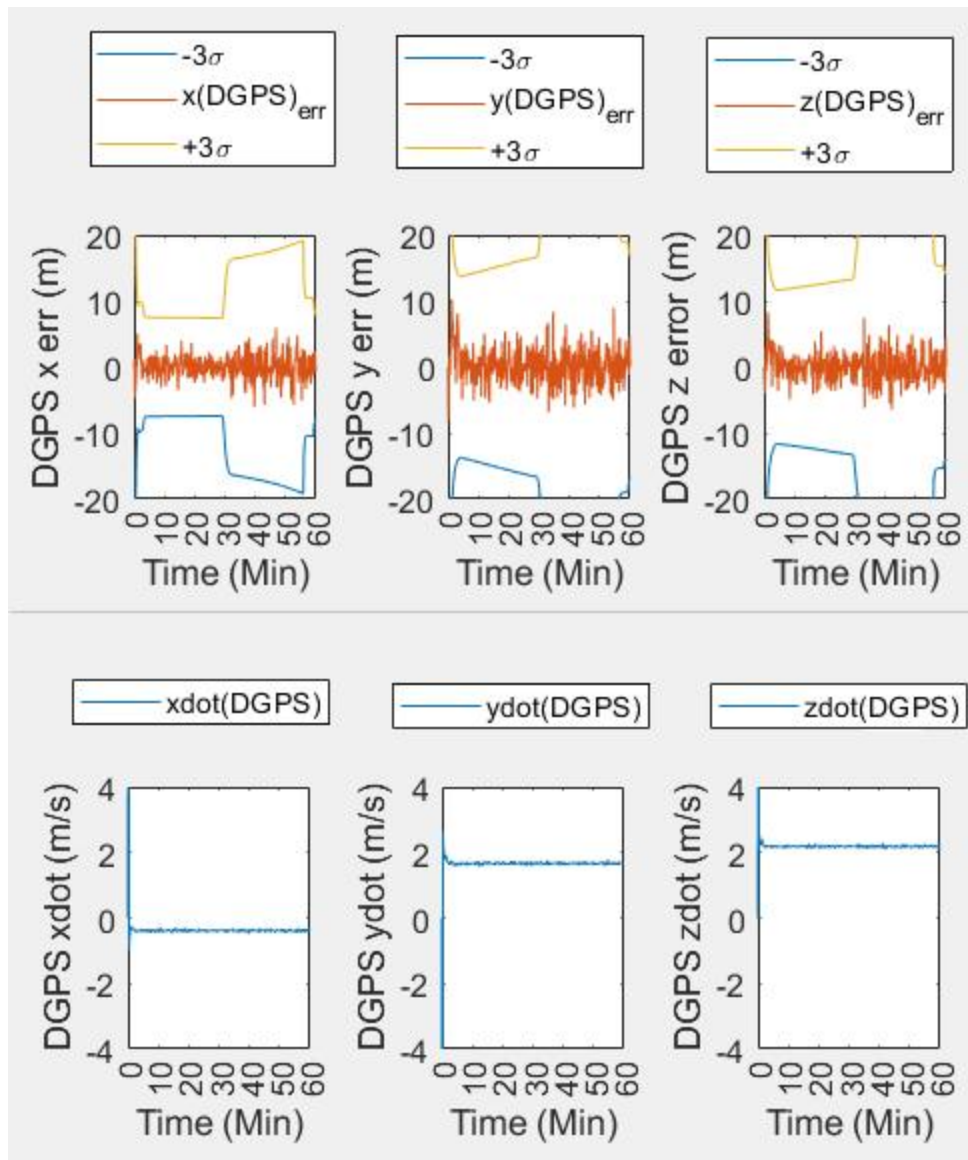
```

문제 2)에서 기술한 POD_HW4.m 파일과 대부분 같은 과정을 거치며 이에 추가로 Commom GPS 위성을 찾기 위해 i와 i -1 사이에서 반복문을 수행하는 과정이 포함된다. 이 과정에 대해 설명한 내용을 첨부하였고 POD_HW4.m과 중복되는 부분은 생략하였다.

7) Explain the results from DGPS.



DGPS의 position vector error를 살펴보면 X방향의 3σ 그래프가 일정하게 유지되는 EKF와 달리 DGPS에서는 Y와 Z방향에서 보이는 양상과 같이 30분을 지나는 시점에서 큰 폭으로 변화한다. 또한 EKF에 비해 전체적으로 error의 폭이 줄어든 것을 볼 수 있다. EKF의 결과에서는 $\pm 10^\circ$ 정도의 error를 보였다면 DGPS에서는 $\pm 5^\circ$ 정도로 감소하였다. DGPS의 경우 위성의 pseudo range 값을 보정하여 오차를 개선하는 방식이기 때문에 EKF를 이용한 결과에 비해 적은 오차가 출력됨을 알 수 있다.



DGPS의 state vector error를 나타낸 그래프이다. 위의 position vector error와 마찬가지로 $\pm 5^\circ$ 정도의 폭을 보이며 error가 발생하고 있다. 3σ 의 변화양상은 이용가능 GPS 수와 관련이 있을 것으로 생각해볼 수 있다. 이에 대한 그림은 다음 페이지에 첨부하였다.

